Práctica 2

Código seguro: del dato al fallo

Puesta en Producción Segura (5023)

1º CE Ciberseguridad en Entornos de las TI

CIPFP Cheste · Curso 2026-27

Nombre y apellidos: Carlos Del Cerro

Fecha: 27 de octubre de 2026

Repositorio: https://github.com/Maximo1001/PPS_1/tree/main

1. Preparación del entorno

Descomprimí el LAB dentro de mi carpeta **pps-arranque**, creé el entorno virtual, lo activé e instalé pytest y coverage. La instalación terminó bien.

```
paolo@paolo:~/Documentos/PPS/pps-arranque$ python3 -m venv .venv
paolo@paolo:~/Documentos/PPS/pps-arranque$ . .venv/bin/activate
(.venv) paolo@paolo:~/Documentos/PPS/pps-arranque$ pip install pytest coverage
Collecting pytest
  Using cached pytest-9.1.1-py3-none-any.whl.metadata (7.6 kB)
Collecting coverage
  Using cached coverage-7.17.0-cp314-cp314-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl.metad
ata (8.1 kB)
Collecting iniconfig>=1.0.1 (from pytest)
  Using cached iniconfig-2.3.1-py3-none-any.whl.metadata (2.8 kB)
Collecting packaging>=22 (from pytest)
  Using cached packaging-26.3-py3-none-any.whl.metadata (3.5 kB)
Collecting pluggy<2,>=1.5 (from pytest)
  Using cached pluggy-1.7.0-py3-none-any.whl.metadata (5.0 kB)
Collecting pygments>=2.7.2 (from pytest)
  Using cached pygments-2.21.0-py3-none-any.whl.metadata (2.5 kB)
Using cached pytest-9.1.1-py3-none-any.whl (386 kB)
Using cached pluggy-1.7.0-py3-none-any.whl (27 kB)
Using cached coverage-7.17.0-cp314-cp314-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl (261 kB
)
Using cached iniconfig-2.3.1-py3-none-any.whl (7.6 kB)
Using cached packaging-26.3-py3-none-any.whl (129 kB)
Using cached pygments-2.21.0-py3-none-any.whl (1.3 MB)
Installing collected packages: pygments, pluggy, packaging, iniconfig, coverage, pytest
Successfully installed coverage-7.17.0 iniconfig-2.3.1 packaging-26.3 pluggy-1.7.0 pygments-2.21.0 pytest-9.1.1
```

Captura 1. Entorno virtual creado e instalación de pytest y coverage.

Después ejecuté el comando para que **.venv**/ no se suba al repositorio. Como no tenía **.gitignore** en esa carpeta, el comando lo creó con esa línea.

```
(.venv) paolo@paolo:~/Documentos/PPS/pps-arranque$ grep -qx '.venv/' .gitignore || echo '.venv/' >> .gitignore
```

Captura 2. Comando que añade .venv/ al .gitignore.

2. Del dato de entrada al sink (PPS-E1)

Ejecuté el validador con un nombre normal y con el dato raro. Con **ana** devolvió True. Con **' OR '1'='1** el programa se paró en la línea 30 de **validar.py** (**return eval(regla)**) con un SyntaxError: el texto que escribí acabó dentro de una orden que Python ejecuta.

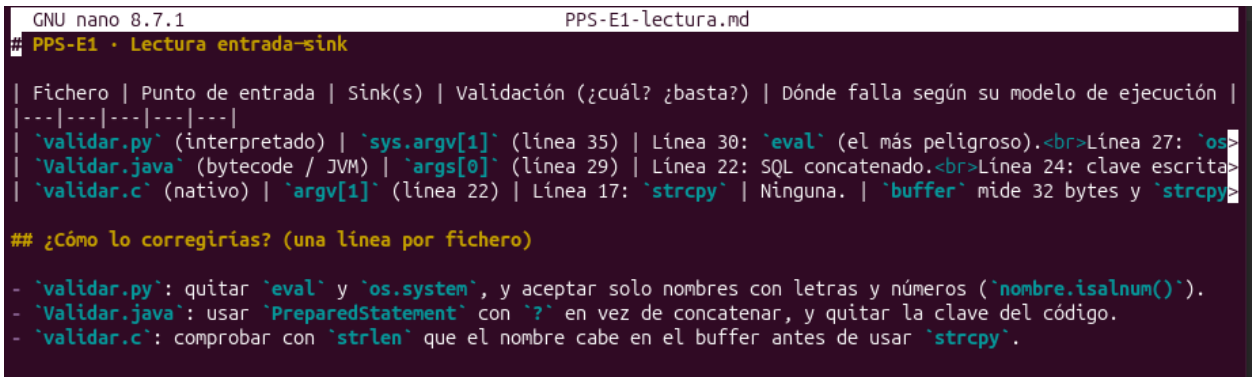
```
(.venv) paolo@paolo:~/Documentos/PPS/pps-arranque/ut1/validador$ python3 validar.py ana
True
(.venv) paolo@paolo:~/Documentos/PPS/pps-arranque/ut1/validador$ python3 validar.py "' OR '1'='1"
Traceback (most recent call last):
  File "/home/paolo/Documentos/PPS/pps-arranque/ut1/validador/validar.py", line 35, in <module>
    print(buscar_usuario(sys.argv[1] if len(sys.argv) > 1 else "ana"))
          ~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/paolo/Documentos/PPS/pps-arranque/ut1/validador/validar.py", line 30, in buscar_usuario
    return eval(regla)
  File "<string>", line 1
    len('' OR '1'='1') > 0
        ^^
SyntaxError: invalid syntax. Is this intended to be part of the string?
```

Captura 3. Ejecución con 'ana' y con "' OR '1'='1".

Leí los tres ficheros (**validar.py**, **Validar.java** y **validar.c**), copié la plantilla y la rellené.

```
(.venv) paolo@paolo:~/Documentos/PPS/pps-arranque/ut1/validador$ cd ..
(.venv) paolo@paolo:~/Documentos/PPS/pps-arranque/ut1$ cp PPS-E1-lectura-PLANTILLA.md PPS-E1-lectura.md
ls
PPS-E1-lectura-PLANTILLA.md  PPS-E1-lectura.md  miniproyecto  validador
```

Captura 4. Copia de la plantilla a PPS-E1-lectura.md.



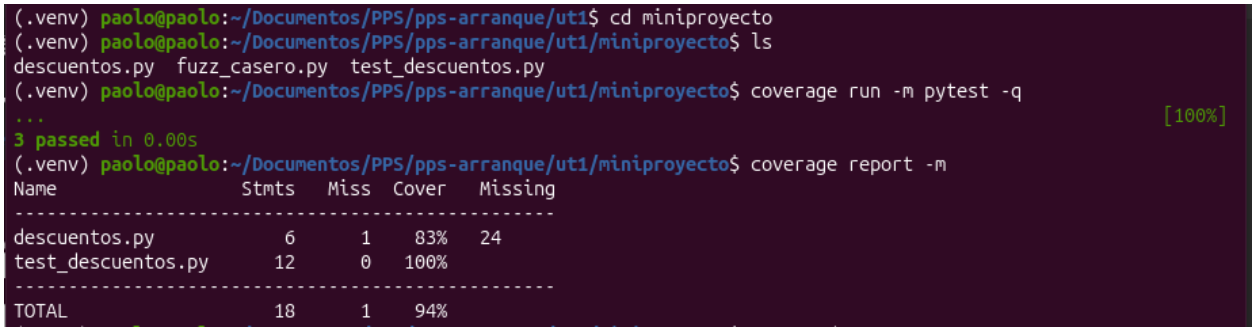
Captura 5. PPS-E1-lectura.md rellenada (editada con nano).

Resumen de lo que encontré:

Fichero	Entrada	Sink	Cómo lo arreglaría
validar.py	sys.argv[1] (línea 35)	eval (línea 30), os.system (27), SQL concatenado (25)	Quitar eval y os.system; aceptar solo letras y números
Validar.java	args[0] (línea 29)	SQL concatenado (22) y clave en el código (24)	PreparedStatement y quitar la clave
validar.c	argv[1] (línea 22)	strcpy (línea 17)	Comprobar con strlen que cabe en el buffer

3. Los tests pasan, pero... (PPS-E2)

Entré en **miniproyecto** y pasé los tests midiendo la cobertura. Pasaron los 3 tests y **descuentos.py** tenía un **83 %** de cobertura; la línea que faltaba era la 24 (**es_precio_valido**).



Captura 6. Tests y cobertura con el fallo todavía dentro.

La incidencia #17 dice que un cupón del 150 % devolvió un precio negativo. Ningún test lo detectaba: probaban descuentos de 10, 0 y 100, y un precio negativo, pero nunca un porcentaje fuera de rango.

4. Test de regresión

Escribí **test_regresion_incidencia_17** al final de **test_descuentos.py**, comprobando que un 150 % y un -5 % lanzan ValueError.

```
  GNU nano 8.7.1                                test_descuentos.py *
"""Suite de PPS-E2. Todo en verde... ¿significa que el código está bien?"""
import pytest
from descuentos import aplicar_descuento, es_precio_valido


# --- ¿Qué comprueba realmente este test?
def test_ejecuta_sin_reventar():
    aplicar_descuento(100, 10)
    assert True


def test_descuento_correcto():
    assert aplicar_descuento(100, 10) == 90.0
    assert aplicar_descuento(50, 0) == 50.0
    assert aplicar_descuento(80, 100) == 0.0


def test_precio_negativo_se_rechaza():
    with pytest.raises(ValueError):
        aplicar_descuento(-10, 10)


# --- TU TEST DE REGRESIÓN (incidencia #17) va aquí debajo.
def test_regresion_incidencia_17():
    with pytest.raises(ValueError):
        aplicar_descuento(100, 150)
    with pytest.raises(ValueError):
        aplicar_descuento(100, -5)
```

Captura 7. Test de regresión escrito en test_descuentos.py.

Lo pasé con la función sin arreglar y mi test falló, como tenía que pasar. El mensaje de pytest fue **Failed: DID NOT RAISE ValueError**.

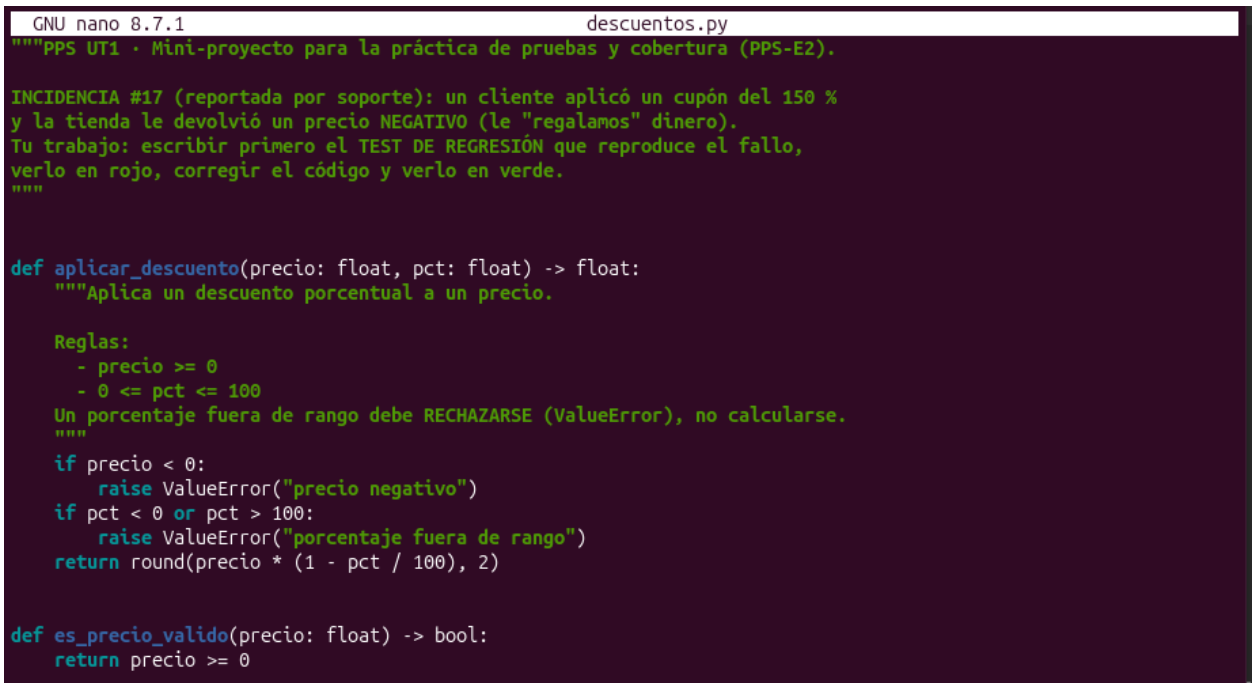
```
(.venv) paolo@paolo:~/Documentos/PPS/pps-arranque/ut1/miniproyecto$ pytest -q
...F                                                                     [100%]
================================================ FAILURES ================================================
_____________________________________ test_regresion_incidencia_17 _____________________________________

    def test_regresion_incidencia_17():
>       with pytest.raises(ValueError):
             ^^^^^^^^^^^^^^^^^^^^^^^^^
E       Failed: DID NOT RAISE ValueError

test_descuentos.py:25: Failed
============================================ short test summary info =============================================
FAILED test_descuentos.py::test_regresion_incidencia_17 - Failed: DID NOT RAISE ValueError
1 failed, 3 passed in 0.02s
```

Captura 8. El test de regresión en rojo.

Arreglé **aplicar_descuento** añadiendo una comprobación del porcentaje (**if pct < 0 or pct > 100: raise ValueError**).



Captura 9. descuentos.py con la comprobación del porcentaje.

Volví a pasar los tests y salieron los 4 en verde.



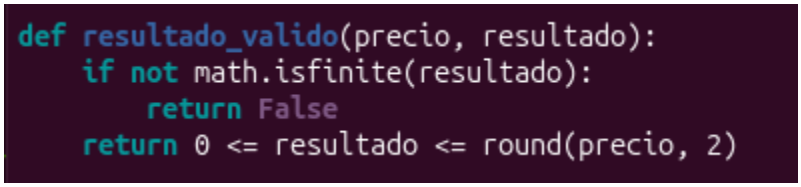
Captura 10. 4 passed.

5. Fuzzing

Ejecuté el fuzzer con la función ya arreglada. Encontró 10 fallos en 20000 intentos, todos con el valor **nan** como precio o como porcentaje:

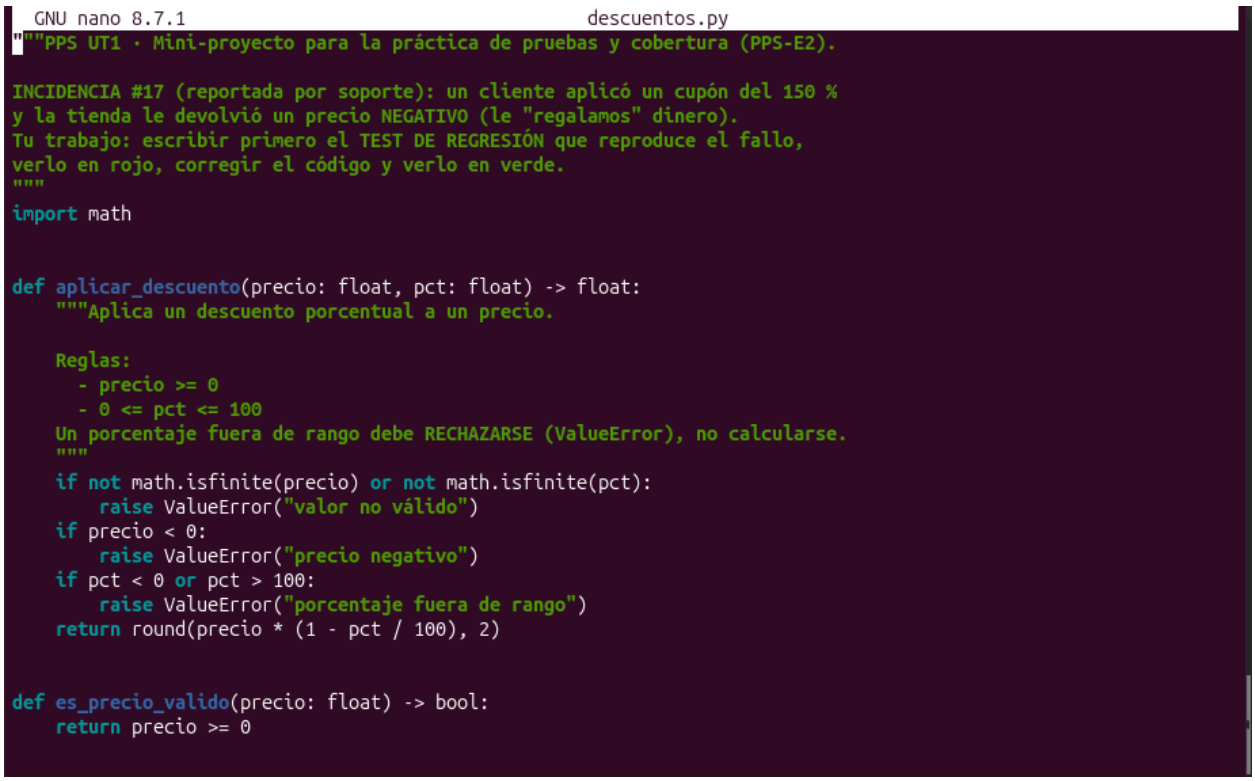
```
$ python3 fuzz_casero.py
RESULTADO RARO nan: precio=861.9135743227646 pct=nan
RESULTADO RARO nan: precio=945.5081878395658 pct=nan
RESULTADO RARO nan: precio=nan pct=100
RESULTADO RARO nan: precio=inf pct=nan
RESULTADO RARO nan: precio=nan pct=0
(...)
10 fallos en 20000 intentos
```

Completé **resultado_valido()** en **fuzz_casero.py**: el resultado tiene que ser un número finito, no negativo y no mayor que el precio (redondeado a 2 decimales).



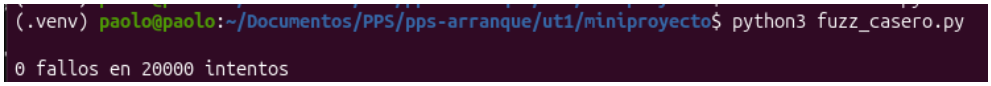
Captura 11. resultado_valido() completada.

Para arreglar **aplicar_descuento** añadí **import math** y una comprobación al principio que rechaza **nan** e **inf** con ValueError.



Captura 12. descuentos.py final.

Con eso el fuzzing dio 0 fallos:



Captura 13. 0 fallos en 20000 intentos.

Por último añadí un test de regresión para lo que encontró el fuzzing, al final de **test_descuentos.py**:

```
def test_regresion_nan_e_infinito():
    with pytest.raises(ValueError):
        aplicar_descuento(100, float("nan"))
    with pytest.raises(ValueError):
        aplicar_descuento(float("nan"), 10)
    with pytest.raises(ValueError):
        aplicar_descuento(float("inf"), 10)
```

Pasé **pytest -q** y salieron **5 passed**. Con el fuzzing en 0 fallos y todos los tests en verde, di la práctica por terminada.

6. Entrega

Comprobé con **git status** que no aparecía **.venv**, y ejecuté:

```
git add ut1
git commit -m "Práctica 2: PPS-E1 y PPS-E2"
git push
```

El primer **git push** fue rechazado (**rejected ... fetch first**) porque mi carpeta apuntaba al repositorio que comparto con un compañero. Cambié el remoto a mi repositorio y volví a subir:

```
git remote set-url origin https://github.com/Maximo1001/PPS_1
git push
(...)
 * [new branch]      main -> main
```

La práctica está subida en la rama **main** de mi repositorio, dentro de la carpeta **ut1**.

Respuestas a las preguntas

1. ¿Cuál es el sink más peligroso de validar.py? ¿Por qué?

eval(regla) (línea 30). Ejecuta como código Python el texto que escribe el usuario, y desde ahí se puede llegar a cualquier otra cosa.

2. ¿Qué cobertura tenía descuentos.py con el fallo dentro?

83 %. Aun así el fallo no se detectaba.

3. ¿Qué mensaje me dio pytest cuando mi test fallaba?

Failed: DID NOT RAISE ValueError.

4. ¿Qué valores encontró el fuzzing que mis tests no habían probado?

nan (como precio y como porcentaje) e **inf** (como precio).

5. URL de mi repositorio.

https://github.com/Maximo1001/PPS_1/tree/main

Observaciones

- Con un 83 % de cobertura y todos los tests en verde, el fallo de la incidencia #17 seguía dentro: los tests no probaban ningún porcentaje fuera de rango.

- El fuzzing encontró **nan** e **inf**, que no se me habían ocurrido. Mis comprobaciones con **<** y **>** no los paraban.

- Al principio mi **resultado_valido()** marcaba como fallo precios redondeados (por ejemplo 824.34 con precio 824.339...). Lo arreglé comparando con **round(precio, 2)**.

- El primer push fue a un repositorio que no era el mío, por eso fue rechazado. Mi práctica está en la rama **main**; el trabajo anterior de Git y Docker sigue en **master**.